

Smart Attendance & Wellbeing System

Face Recognition-Based Attendance with Emotion Analytics

Project Report

Course	Computer Vision
Submission Type	Flipped Course Evaluation -- Build Your Own Project
Repository	<a href="https://github.com/<your-username>/<your-repo>">https://github.com/<your-username>/<your-repo>
Submission Date	September 2026

2. Introduction

This report documents the design and implementation of a command-line Computer Vision system that automates attendance marking using face recognition and enriches that record with a lightweight, per-check-in emotion estimate. The project applies core Computer Vision techniques covered in the course -- Haar-cascade face detection, Local Binary Pattern Histogram (LBPH) face recognition, and Histogram-of-Oriented-Gradients (HOG) feature extraction with an SVM classifier -- inside a single, modular, fully terminal-operable application.

The system is organized into three functional modules (Enrollment, Attendance, and Analytics/Reporting), backed by a SQLite database, and is accompanied by an automated test suite that validates each module using deterministic synthetic data.

3. Problem Statement

Manual attendance tracking (roll calls, sign-in sheets) is slow, easy to falsify, and provides no insight beyond a bare present/absent record. Small classrooms, workshops, or office teams need a lightweight way to confirm attendance using something harder to fake than a signature, and to get a rough, non-intrusive sense of group wellbeing over time, without adopting a heavyweight commercial biometric platform.

This project addresses that need with a self-hosted, CLI-driven system: it enrolls a known roster of people by face, automatically recognizes and logs attendance from a photo or webcam feed, and tags each check-in with a coarse emotion label -- giving a facilitator both an attendance register and a rough wellbeing trend line.

4. Functional Requirements

The system implements three major functional modules:

- **Module 1 -- Enrollment:** Register a new person from a batch of photos or a live webcam session. Input: name + images/webcam stream. Output: normalized face images saved to `data/known_faces/<name>/` and a new row in the `users` table.
- **Module 2 -- Attendance:** Detect and recognize faces in an image, video frame, or webcam stream; classify emotion; log one attendance record per person per day. Input: image path or webcam feed. Output: console summary + a row in the `attendance` table.
- **Module 3 -- Analytics & Reporting:** Export attendance data to CSV, compute a per-person summary (days present, dominant emotion), and render attendance-trend and emotion-distribution charts. Input: date range (optional). Output: CSV file(s) and PNG chart(s).

All three modules are reachable through a single CLI entry point (`main.py`), giving the system a clear, linear workflow: `enroll` → `train` → `mark attendance` → `report`.

5. Non-Functional Requirements

Requirement	How it is addressed
-------------	---------------------

Performance	LBPH and HOG+SVM run on CPU in milliseconds/seconds; no GPU or network calls at inference time
Reliability	A DB uniqueness constraint prevents duplicate attendance; emotion-model failures degrade gracefully
Usability	Single CLI with -h help text on every subcommand; clear [OK]/[ERROR] console output.
Maintainability	Modular package layout (src/), one responsibility per file, all configuration centralized in config.py.
Error Handling	Custom exceptions (EnrollmentError, AnalyticsError) with descriptive messages instead of raw stack
Logging / Monitoring	Rotating file logger (app.log) plus console output for every recognition/attendance/training event.
Scalability	SQLite schema supports an arbitrary number of users/records; swapping to a server DB only requires

6. System Architecture

The system is layered into a CLI layer, three core functional modules, a Computer Vision layer (detection / recognition / emotion), and a storage layer, as shown below.

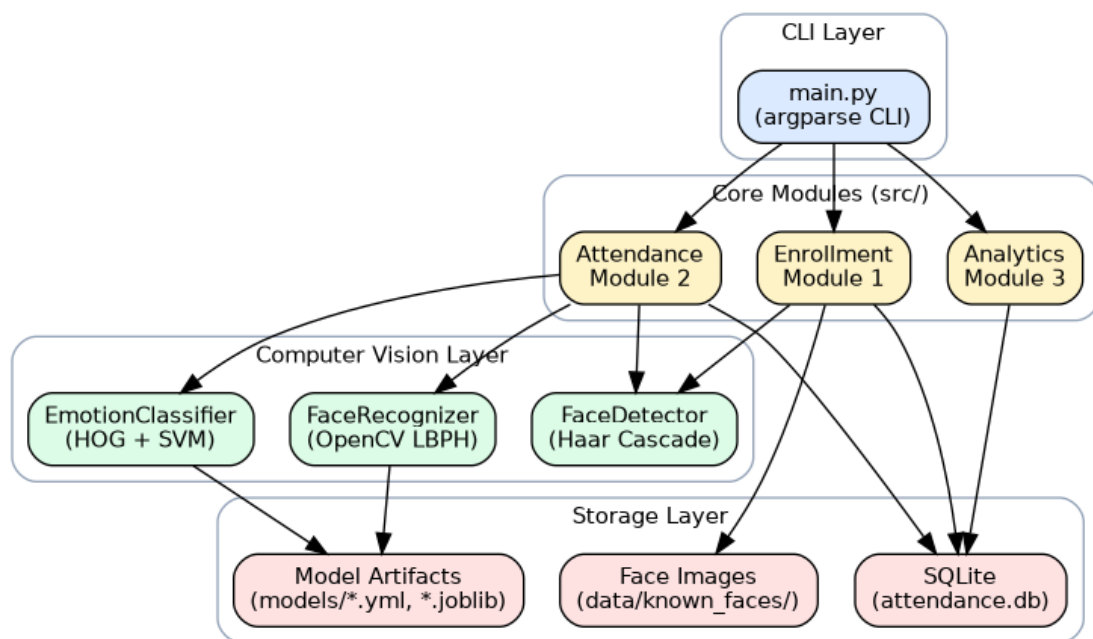


Figure 1: System architecture

7. Design Diagrams

7.1 Use Case Diagram

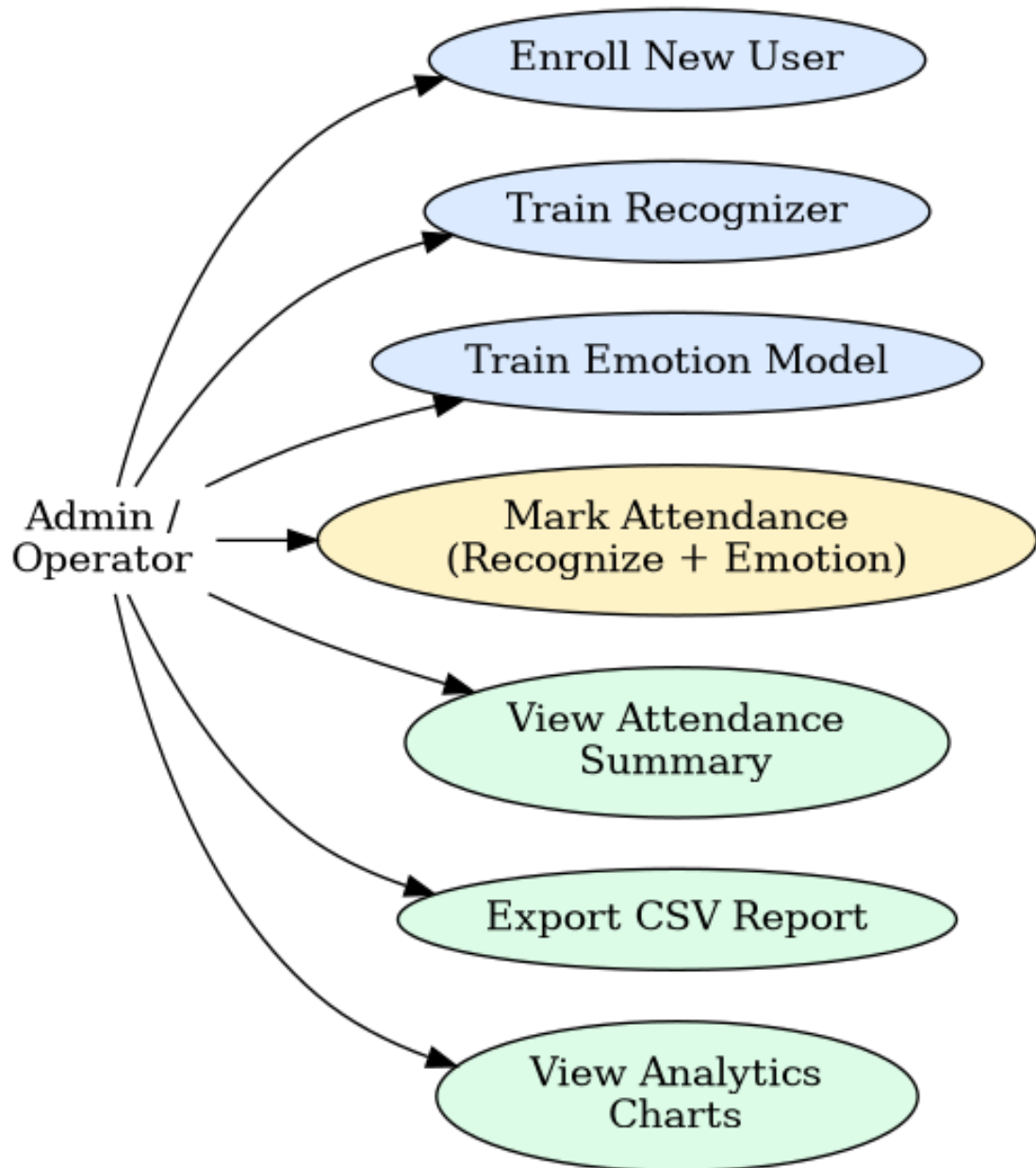


Figure 2: Use case diagram

7.2 Workflow / Process Flow Diagram



Figure 3: End-to-end workflow

7.3 Sequence Diagram -- Attendance Marking Flow

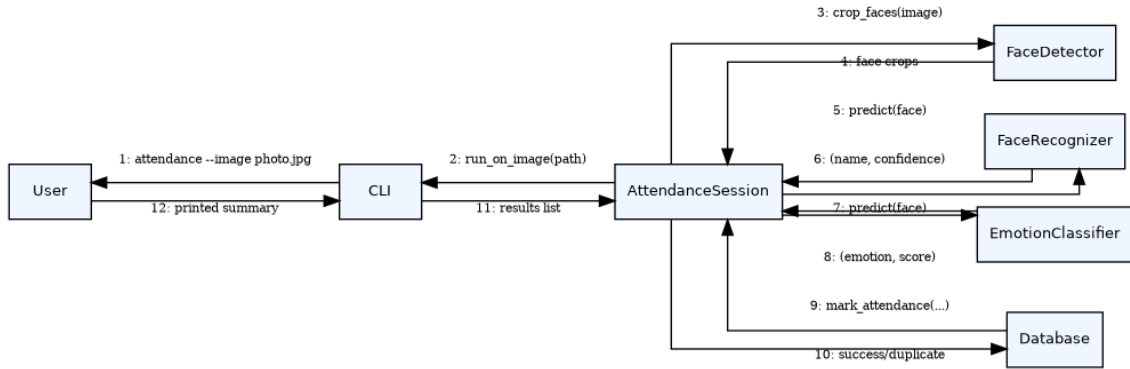


Figure 4: Sequence diagram for a single attendance check

7.4 Class / Component Diagram

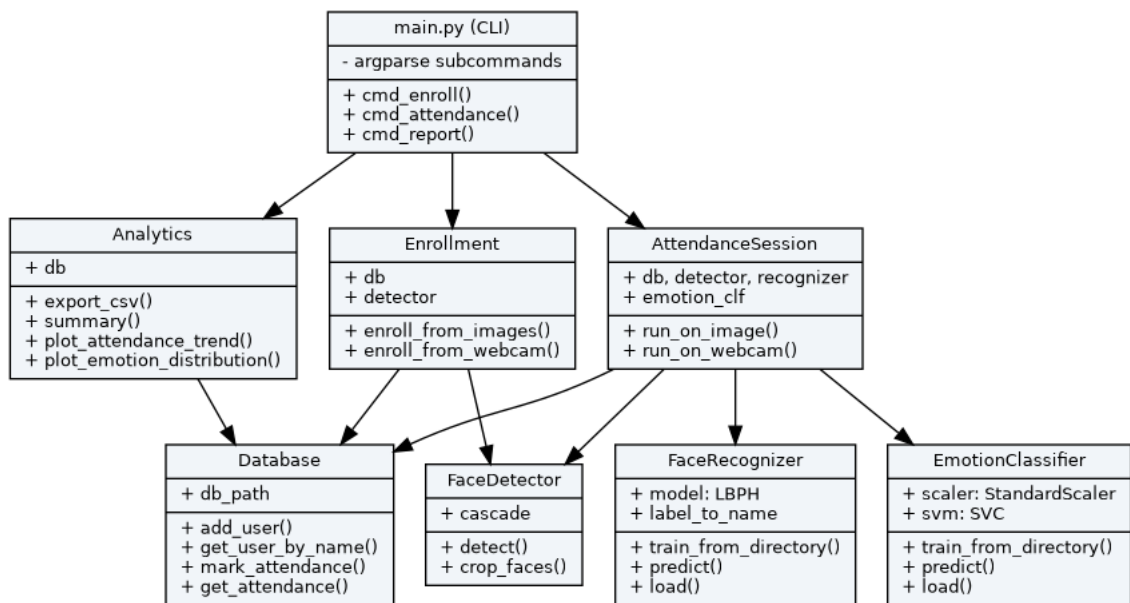


Figure 5: Class/component diagram

7.5 ER Diagram / Schema Design

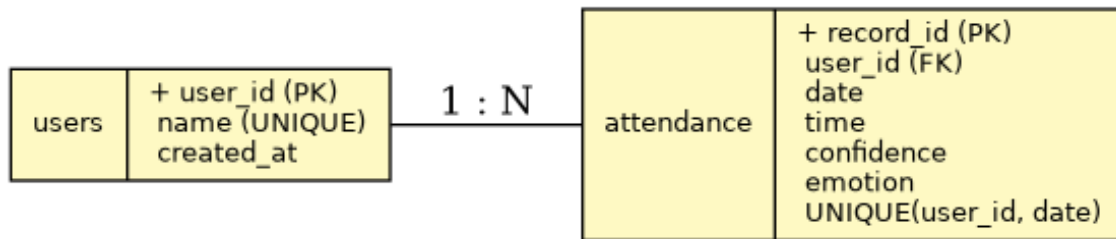


Figure 6: Entity-relationship diagram

users(`user_id` PK, `name` UNIQUE, `created_at`) — one row per enrolled person.

attendance(`record_id` PK, `user_id` FK, `date`, `time`, `confidence`, `emotion`, `UNIQUE(user_id, date)`) — one row per person per day, enforced by the database itself.

8. Design Decisions & Rationale

8.1 Face recognition: OpenCV LBPH vs. deep embeddings

LBPH was chosen over deep-embedding approaches (FaceNet/ArcFace) because the target deployment is a small, controlled roster (5-20 photos per person) run from the command line with no guaranteed GPU. LBPH needs no pretrained-weight download, trains in seconds on CPU, and is a classical feature-based technique central to this course's syllabus. The trade-off is reduced robustness to large pose/illumination variation compared to deep models -- acceptable for a controlled indoor camera, and listed under Future Enhancements.

8.2 Emotion recognition: HOG + SVM vs. a CNN

Similarly, HOG+SVM trains in seconds on CPU using `scikit-learn/scikit-image`, with no GPU and no large pretrained model, making the whole project installable with a single `pip install -r requirements.txt`. The `EmotionClassifier` class exposes a stable `train_from_directory()` / `predict()` interface, so a future CNN-based implementation could be substituted without changing any calling code.

8.3 Storage: SQLite vs. a client-server database

SQLite ships with Python's standard library, needs no server process, and is more than sufficient for a single-roster attendance log, keeping the setup to one `pip install`.

9. Implementation Details

The codebase follows a strict one-module-one-responsibility layout:

File	Responsibility
config.py	Central configuration: all paths and tunable parameters
src/db.py	SQLite data-access layer (users, attendance)
src/face_detector.py	Haar-cascade face detection wrapper
src/face_recognizer.py	LBPH training / inference / persistence
src/emotion_model.py	HOG feature extraction + SVM training / inference
src/enrollment.py	Functional Module 1 -- enrollment from images or webcam
src/attendance.py	Functional Module 2 -- recognition + attendance + emotion logging
src/analytics.py	Functional Module 3 -- CSV export, summaries, charts
src/utils/	Logging and image-processing helper functions
main.py	argparse-based CLI tying every module together
tests/	pytest suite with a synthetic, seeded data generator

Key implementation choices: face images are always converted to grayscale and histogram-equalized before recognition/emotion inference to reduce lighting sensitivity; LBPH confidence is a distance score, so predictions above `RECOGNITION_CONFIDENCE_THRESHOLD` are relabeled 'Unknown' rather than a wrong name; attendance writes are guarded by a database `UNIQUE` constraint rather than application-level checks, so a race between two near-simultaneous recognitions cannot double-log a person.

10. Screenshots / Results

Representative CLI sessions demonstrating the system's usability and error handling:

```
terminal - smart-attendance

$ python main.py -h
usage: smart-attendance [-h]
                        {enroll,train-recognizer,train-emotion,attendance,report,list-users}
                        ...

Smart Attendance & Wellbeing System -- face recognition + emotion analytics
from the CLI.

positional arguments:
  {enroll,train-recognizer,train-emotion,attendance,report,list-users}
  enroll                Register a new person (Module 1)
  train-recognizer      Train the LBPH face recognizer on enrolled faces
  train-emotion         Train the HOG+SVM emotion classifier
  attendance            Run recognition + mark attendance (Module 2)
  report               Generate analytics/reports (Module 3)
  list-users            List all enrolled users

options:
  -h, --help            show this help message and exit
```

Figure 7: CLI help output listing all subcommands

```
terminal - smart-attendance

$ python main.py list-users
#1  Alice
#2  Bob
#3  Charlie
#4  Diana
```

Figure 8: Listing enrolled users

```
terminal - smart-attendance

$ python main.py report --summary
name  days_present  dominant_emotion
Alice      4          angry
Bob        7          angry
Charlie    7          happy
Diana      6          angry
```

Figure 9: Per-person attendance/emotion summary

```
terminal - smart-attendance

$ python main.py enroll --name "Test" --images "nonexistent_*.jpg"
[ERROR] At least one image is required for enrollment.
```

Figure 10: Graceful error handling when no face is detected during enrollment

Generated analytics charts from a seeded 7-day demo dataset (see scripts/seed_demo_data.py):

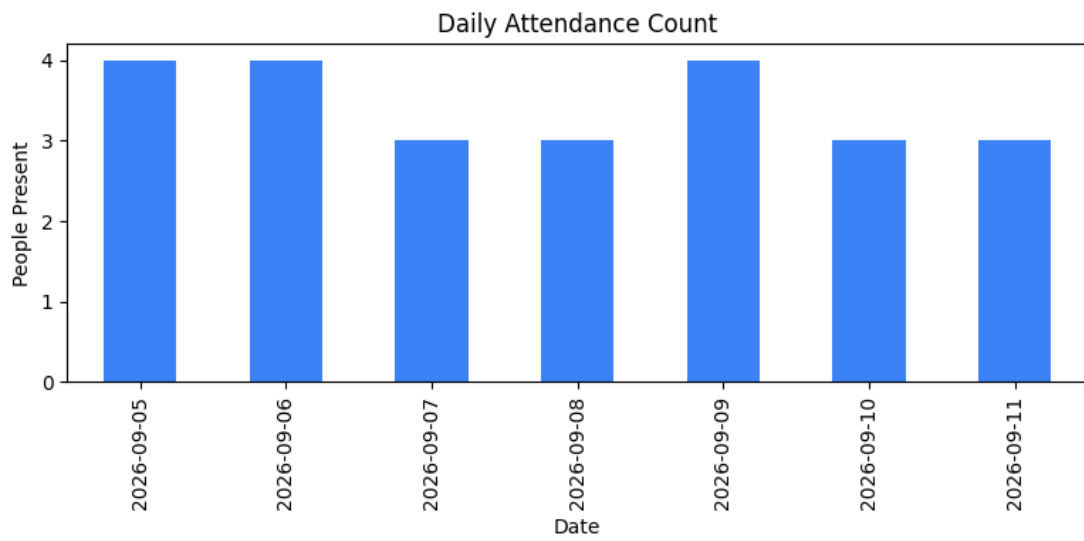


Figure 11: Daily attendance trend

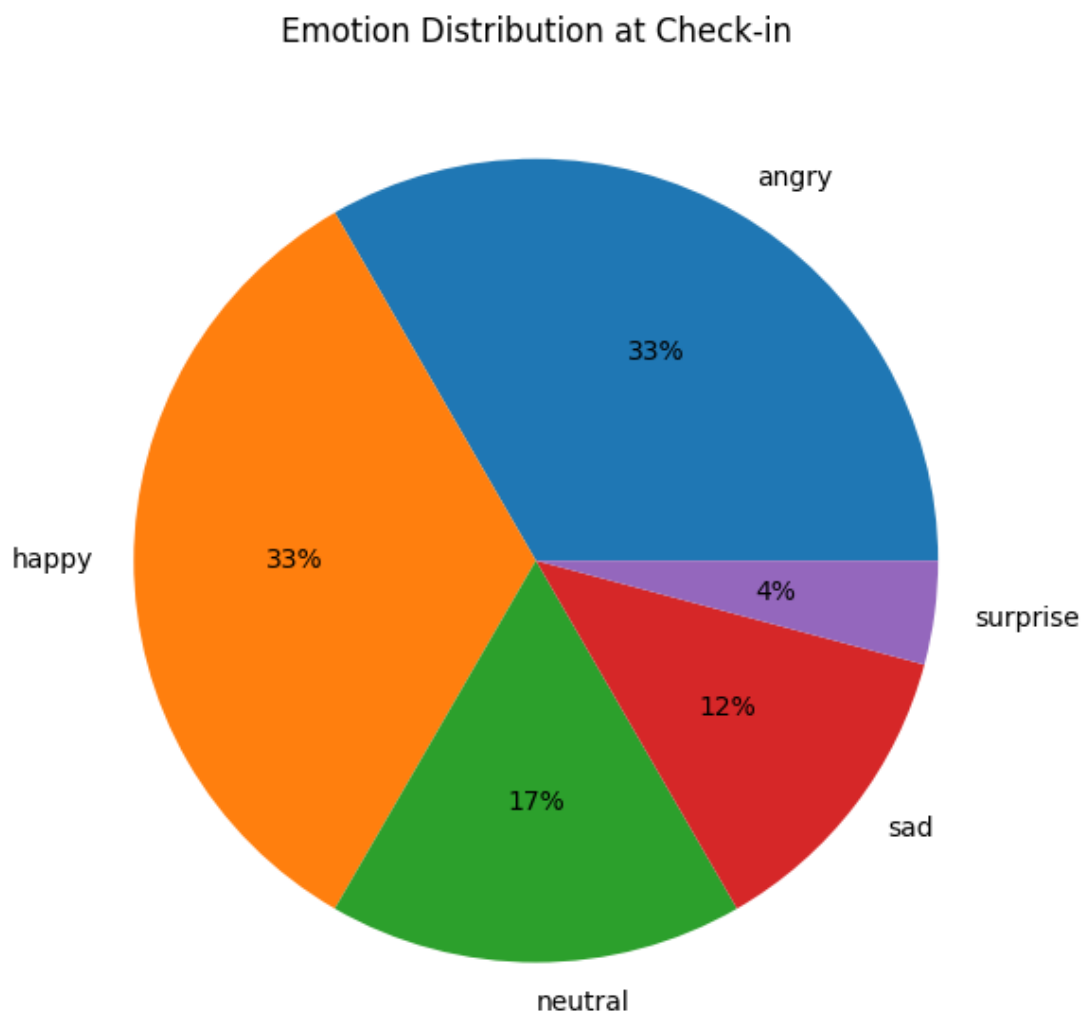


Figure 12: Emotion distribution at check-in

11. Testing Approach

The project uses `pytest` with a fully synthetic, seeded data generator (`tests/conftest.py`) so the entire suite runs without a webcam or any external dataset -- important for reproducible grading. Coverage includes:

- **Database layer:** user creation/lookup, duplicate-attendance rejection, date-range filtering.
- **Face recognizer:** trains on two synthetic, class-separable pattern sets and verifies correct discrimination between them, plus a `ValueError` when trained on an empty directory.
- **Emotion classifier:** trains on a 3-class synthetic dataset, verifies persistence (train → save → reload → predict), and rejects single-class training data.
- **Analytics:** CSV export, per-person summary correctness, and chart-file generation, including the empty-dataset error path.
- **Enrollment:** input validation (empty name / no images), no-face-detected abort, and a mocked-detector happy path verifying file save + DB registration logic in isolation.

All persistence-touching tests run against a temporary directory via a `tmp_config` `pytest` fixture, so running the suite never touches real enrollment data or the production database.

12. Challenges Faced

- **Choosing lightweight models without sacrificing correctness:** the initial instinct was to reach for deep-learning face embeddings and a CNN emotion classifier, but that would have made the 'runs from a single pip install, no GPU required' requirement much harder to satisfy. Settling on LBPH and HOG+SVM required re-validating that they were still faithful to the course's classical-CV concepts rather than a step down in rigor.
- **Avoiding double attendance marks:** an application-level 'check then insert' approach is vulnerable to race conditions; moving the uniqueness rule into the database schema itself (a `UNIQUE` constraint) made the guarantee robust regardless of calling code.
- **Testing computer-vision code without a webcam or real face dataset:** solved by building a deterministic synthetic pattern-image generator that still exercises the real training/prediction code paths for both LBPH and HOG+SVM.
- **Keeping emotion tagging from ever breaking attendance:** emotion prediction is wrapped so that any failure (e.g. a missing/corrupt model file) logs a warning and continues, rather than aborting the more important attendance log.

13. Learnings & Key Takeaways

- Classical Computer Vision techniques (Haar cascades, LBPH, HOG+SVM) remain highly practical for small, controlled-deployment problems, and are far cheaper to set up and run than deep learning alternatives.
- Pushing data-integrity guarantees (like 'one attendance record per person per day') down into the database schema is more robust than enforcing them only in application code.
- A clean module boundary between detection, recognition, and classification (rather than one monolithic 'vision.py') made it straightforward to unit-test each piece independently and to reason about where a future upgrade (e.g. a CNN) would plug in.
- Designing for CLI-only, headless execution from the start (rather than retrofitting it later) meaningfully shaped the architecture -- e.g. every webcam-based function has a static-image equivalent, which is also what made automated testing possible at all.

14. Future Enhancements

- Replace LBPH with a deep face-embedding model (e.g. ArcFace) behind the same FaceRecognizer interface, for better robustness to pose and lighting variation.
- Replace HOG+SVM with a CNN trained on the full FER2013 dataset for higher emotion-classification accuracy.
- Add a `--train-test-split` evaluation flag reporting accuracy and a confusion matrix for both models.
- Add basic liveness/anti-spoofing detection to prevent a printed photo from being recognized.
- Wrap the existing modules in a small REST API for integration with a web-based dashboard, without changing any core logic.

15. References

- Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face description with local binary patterns: Application to face recognition. *IEEE TPAMI*.
- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. *CVPR*.
- Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *CVPR*.
- Goodfellow, I. et al. (2013). Challenges in representation learning: A report on three machine learning contests (origin of the FER2013 dataset).
- OpenCV documentation: <https://docs.opencv.org/>
- scikit-learn documentation: <https://scikit-learn.org/>
- scikit-image HOG documentation: <https://scikit-image.org/docs/stable/api/skimage.feature.html>

